

Dependency, Interval, Bit, Math & Design

Pattern yang muncul ketika masalah punya urutan ketergantungan, rentang, operasi bit, observasi matematika, atau desain struktur data.

Tujuan dokumen ini: bukan menghafal tanda-tanda soal, tapi memahami gerak berpikir pattern. Setiap pattern dijelaskan dengan contoh dunia nyata, contoh teknologi, konsep komputasi, alasan kenapa contoh itu sama dengan pattern, dan template Dart minimal.

Cara Membaca Dokumen Ini

Jangan baca seperti kamus keyword. Baca seperti latihan membangun intuisi. Pattern LeetCode itu mirip alat kerja: tiap alat punya cara bergerak, situasi natural, dan bentuk data yang cocok.

Untuk tiap pattern, pakai urutan baca ini: makna manusiawi -> contoh dunia nyata -> contoh teknologi -> konsep komputasi -> bentuk LeetCode -> template Dart -> pertanyaan cek pemahaman.

Pattern	Makna inti
Topological Sort	mengurutkan dependency tanpa melanggar prasyarat
Intervals	mengelola rentang yang overlap
Bit Manipulation	memakai bit sebagai status/operasi cepat
Math	observasi angka dan rumus
Design Problems	menggabungkan struktur data untuk operasi tertentu

Topological Sort - mengurutkan dependency tanpa melanggar prasyarat

Makna manusiawinya

Topological sort mencari urutan valid untuk hal-hal yang punya prasyarat. Sesuatu baru bisa dikerjakan setelah dependency-nya selesai.

Contoh nyata non-teknologi

Belajar kurikulum

Lu tidak bisa belajar advanced DP sebelum recursion dan array cukup kuat. Materi punya dependency.

Memasak beberapa menu

Nasi harus dimasak sebelum plating. Saus harus selesai sebelum dicampur. Ada urutan yang harus dipatuhi.

Contoh nyata di dunia teknologi

Build system

File/module dibangun sesuai dependency. Modul yang tidak punya prasyarat bisa mulai dulu.

Course Schedule

Course hanya bisa diambil setelah prerequisite selesai.

Konsep komputasinya

Secara komputasi, topological sort bisa memakai Kahn algorithm dengan indegree atau DFS cycle detection. Kalau ada cycle, tidak ada urutan valid.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena semua punya prasyarat. Indegree 0 artinya pekerjaan bisa dimulai sekarang.

Bentuknya di LeetCode

- Course Schedule I/II
- Alien Dictionary
- Build order
- Task dependency
- Cycle detection directed graph

Template Dart minimal

```
import 'dart:collection';

bool canFinish(int n, List<List<int>> prerequisites) {
  final graph = List.generate(n, (_) => <int>[]);
  final indegree = List.filled(n, 0);

  for (final p in prerequisites) {
    final course = p[0], pre = p[1];
    graph[pre].add(course);
    indegree[course]++;
  }
}
```

```

final queue = Queue<int>();
for (int i = 0; i < n; i++) if (indegree[i] == 0) queue.add(i);

int taken = 0;
while (queue.isNotEmpty()) {
  final node = queue.removeFirst();
  taken++;
  for (final next in graph[node]) {
    indegree[next]--;
    if (indegree[next] == 0) queue.add(next);
  }
}
return taken == n;
}

```

Kalau masih bingung, tanya begini

- Apa node dan dependency-nya?
- Apakah directed?
- Apa arti indegree 0?
- Apakah cycle membuat mustahil?

Mini drill pemahaman

1. Kenapa dependency harus directed?
2. Apa yang terjadi jika A butuh B dan B butuh A?
3. Buat topological order untuk belajar LeetCode.

Intervals - mengelola rentang yang overlap

Makna manusiawinya

Interval adalah rentang start-end. Pattern ini muncul saat kita harus menggabungkan, memilih, menghapus, atau menghitung konflik antar rentang.

Contoh nyata non-teknologi

Jadwal meeting

Meeting 10-11 dan 10:30-12 overlap. Harus digabung sebagai busy time atau butuh ruangan berbeda.

Booking hotel

Dua booking overlap kalau tanggal masuk satu tamu sebelum tanggal keluar tamu lain.

Contoh nyata di dunia teknologi

Calendar app

Detect conflict dan merge availability.

Video timeline

Subtitle, highlight, atau ads punya start-end time yang bisa overlap.

Konsep komputasinya

Secara komputasi, interval hampir selalu dimulai dengan sort berdasarkan start atau end. Setelahurut, overlap bisa dicek lokal dengan interval terakhir.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena rentang perlu dibandingkan dari posisi awal/akhir. Sorting mengubah masalah global menjadi scan linear.

Bentuknya di LeetCode

- Merge Intervals
- Insert Interval
- Meeting Rooms
- Non-overlapping Intervals
- Minimum Arrows

Template Dart minimal

```
List<List<int>> mergeIntervals(List<List<int>> intervals) {
  intervals.sort((a, b) => a[0].compareTo(b[0]));
  final merged = <List<int>>[];

  for (final interval in intervals) {
    if (merged.isEmpty || interval[0] > merged.last[1]) {
      merged.add([interval[0], interval[1]]);
    } else {
      merged.last[1] = merged.last[1] > interval[1] ? merged.last[1] : interval[1];
    }
  }
  return merged;
}
```

Kalau masih bingung, tanya begini

- Sort berdasarkan start atau end?
- Apakah perlu merge, count, atau remove?
- Apa definisi overlap?
- Apakah interval tertutup/terbuka?

Mini drill pemahaman

1. Kenapa sort membuat merge interval mudah?
2. Apa beda meeting rooms dan merge intervals?
3. Kapan greedy pilih end time?

Bit Manipulation - memakai bit sebagai status/operasi cepat

Makna manusiawinya

Bit manipulation memakai representasi biner untuk menyimpan status on/off atau melakukan operasi matematika rendah-level dengan cepat.

Contoh nyata non-teknologi

Saklar lampu

Setiap bit seperti saklar: 1 menyala, 0 mati. Satu angka bisa menyimpan banyak status.

Checklist kecil

Status 10 fitur aktif/nonaktif bisa disimpan sebagai 10 bit.

Contoh nyata di dunia teknologi

Permission flags

Read/write/execute bisa direpresentasikan sebagai bit flag.

Subset mask

Untuk n kecil, setiap subset bisa direpresentasikan oleh angka bitmask.

Konsep komputasinya

Secara komputasi, operasi penting: XOR, AND, OR, shift. XOR punya sifat $x \oplus x = 0$ dan $x \oplus 0 = x$, berguna untuk mencari single number.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena setiap bit membawa status kecil. Operasi bit memungkinkan cek, tambah, hapus, dan toggle status dengan cepat.

Bentuknya di LeetCode

- Single Number
- Counting Bits
- Subsets with bitmask
- Missing Number
- Reverse Bits

Template Dart minimal

```
int singleNumber(List<int> nums) {  
  int result = 0;  
  for (final x in nums) {  
    result ^= x;  
  }  
  return result;  
}
```

Kalau masih bingung, tanya begini

- Apakah status bisa on/off?
- Apakah XOR menghapus pasangan?
- Apakah n kecil sehingga subset bitmask masuk akal?

Mini drill pemahaman

1. Kenapa $x \oplus x$ jadi 0?
2. Modelkan permission read/write sebagai bit.
3. Kapan bitmask lebih simple daripada list boolean?

Math - observasi angka dan rumus

Makna manusiawinya

Math pattern muncul ketika solusi utama bukan struktur data rumit, tapi observasi angka: modulo, gcd, prime, combinatorics, geometry, fast exponentiation.

Contoh nyata non-teknologi

Membagi jadwal berulang

Kalau event berulang setiap 7 hari, modulo membantu menentukan hari yang sama.

Menyederhanakan rasio

GCD membantu menyederhanakan 12:18 menjadi 2:3.

Contoh nyata di dunia teknologi

Hashing/modulo buckets

Modulo dipakai untuk distribusi bucket.

Cryptography dasar

Banyak operasi modern bergantung pada aritmetika modular, meski LeetCode biasanya versi sederhana.

Konsep komputasinya

Secara komputasi, math pattern sering mengubah brute force menjadi formula. Fast exponentiation memecah pangkat besar menjadi kuadrat berulang.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena angka punya struktur. Kalau struktur ini terlihat, kita tidak perlu simulasi langkah demi langkah.

Bentuknya di LeetCode

- Pow(x,n)
- Sqrt
- Happy Number
- GCD
- Prime counting
- Rotate Image
- Spiral Matrix

Template Dart minimal

```
int gcd(int a, int b) {  
  while (b != 0) {  
    final temp = a % b;  
    a = b;  
    b = temp;  
  }  
  return a.abs();  
}
```

```
int fastPowInt(int x, int n) {  
    int result = 1;  
    int base = x;  
    while (n > 0) {  
        if (n.isOdd) result *= base;  
        base *= base;  
        n ~/= 2;  
    }  
    return result;  
}
```

Kalau masih bingung, tanya begini

- Apakah ada rumus yang menyederhanakan?
- Apakah modulo/gcd relevan?
- Apakah brute force terlalu besar tapi pola angka jelas?

Mini drill pemahaman

1. Kenapa fast pow $O(\log n)$?
2. Kapan modulo membantu?
3. Apa bedanya math observation dan DP?

Design Problems - menggabungkan struktur data untuk operasi tertentu

Makna manusiawinya

Design problem meminta kita merancang struktur agar operasi tertentu cepat. Biasanya jawabannya bukan satu pattern, tapi kombinasi beberapa struktur data.

Contoh nyata non-teknologi

Mendesain meja kerja

Kalau lu sering ambil pulpen, taruh pulpen dekat. Kalau dokumen jarang dipakai, taruh jauh. Desain mengikuti operasi yang sering terjadi.

Sistem arsip

Butuh cepat cari dokumen dan cepat tahu dokumen terbaru. Mungkin perlu index plus urutan waktu.

Contoh nyata di dunia teknologi

LRU Cache

HashMap untuk akses cepat, doubly linked list untuk urutan recent.

TimeMap

HashMap key -> list timestamp, lalu binary search untuk timestamp terdekat.

Konsep komputasinya

Secara komputasi, design dimulai dari daftar operasi dan target complexity. Pilih struktur data yang menutup kebutuhan tiap operasi.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena alat disusun berdasarkan cara dipakai. Tidak ada struktur data universal; desain bagus adalah tradeoff sadar.

Bentuknya di LeetCode

- LRU Cache
- Min Stack
- Insert Delete GetRandom O(1)
- Time Based Key-Value Store
- Design Twitter

Template Dart minimal

```
class TimeMap {
  final Map<String, List<(int, String)>> store = {};

  void set(String key, String value, int timestamp) {
    store.putIfAbsent(key, () => []).add((timestamp, value));
  }

  String get(String key, int timestamp) {
    final arr = store[key];
```

```

    if (arr == null) return '';
    int l = 0, r = arr.length - 1;
    String ans = '';
    while (l <= r) {
        final m = (l + r) ~/ 2;
        if (arr[m].$1 <= timestamp) {
            ans = arr[m].$2;
            l = m + 1;
        } else {
            r = m - 1;
        }
    }
    return ans;
}
}

```

Kalau masih bingung, tanya begini

- Operasinya apa saja?
- Target complexity tiap operasi apa?
- Struktur data mana yang menutup operasi mana?
- Apa tradeoff memory?

Mini drill pemahaman

1. Untuk LRU, kenapa HashMap saja tidak cukup?
2. Untuk TimeMap, kenapa list timestamp harus terurut?
3. Tuliskan operasi sebelum memilih struktur.

Penutup

Mastery bukan berarti menghafal semua tanda soal. Mastery berarti saat melihat masalah, lu bisa memodelkan gerak masalahnya: apakah ia menyebar, menyelam, menjaga window, mencari boundary, mengingat key, menggabungkan kelompok, atau menyimpan subproblem.

Cara latihan: ambil satu pattern, pahami analoginya, kerjakan 3-5 soal mudah, tulis ulang konsepnya dengan bahasa sendiri, lalu campur dengan pattern lain supaya otak belajar mengenali bentuk masalah tanpa spoiler.